

forward-biased and supplies power to the load, while the diode on the AC-derived DC side remains reverse-biased.

At night or in low-light conditions, when solar voltage falls below the AC-derived DC voltage, the diode on the DC side conducts and powers the load, while the solar-side

diode becomes reverse-biased. This arrangement ensures seamless and automatic source selection.

2. Generation side control. The ESP32 WROVER module controls the generation side of this hybrid smart grid system. The sensors used include ACS712 for measuring Generator 1 current, Generator 2

current, and detecting earth faults; ZMPT101B for AC voltage measurement; PC718 for frequency measurement; and INA219 for monitoring DC load and performing load balancing.

A 5V relay disconnects the circuit in case of a short circuit, overvoltage, overfrequency, underfrequency, or an earth fault. All sensor data is transmitted to the Firebase Realtime Database for cloud-based monitoring and control.

Commands are sent from Firebase to the load side for automatic or manual load management. In the Firebase dashboard, users can control essential and non-essential loads while monitoring voltage, current, frequency, and power in real time.

3. Load side control. The ESP8266 module controls this section. It receives remote control commands through Firebase IoT and operates 5V relays to connect or disconnect loads either automatically or manually.

For assembly, the generator-side circuit should be connected as shown in Fig. 3, followed by the load-side circuit as shown in Fig. 4.

Note. Both ESP boards and the relay modules can be powered using a 5V DC USB adaptor.

Software

Both sides of the code are created in the Arduino IDE. Begin by setting up the IDE and installing the ESP board package. The installation instructions are available at:

<https://docs.espressif.com/projects/arduino-esp32/en/latest/installing.html>

Next, install the libraries Firebase_ESP_Client, Adafruit_INA219, and FirebaseESP8266 using the Library Manager in the Tools section. After installation, Firebase can be prepared for data handling.

Firestore configuration

Step 1: Create a Firestore Project

1. Open Firestore Console:

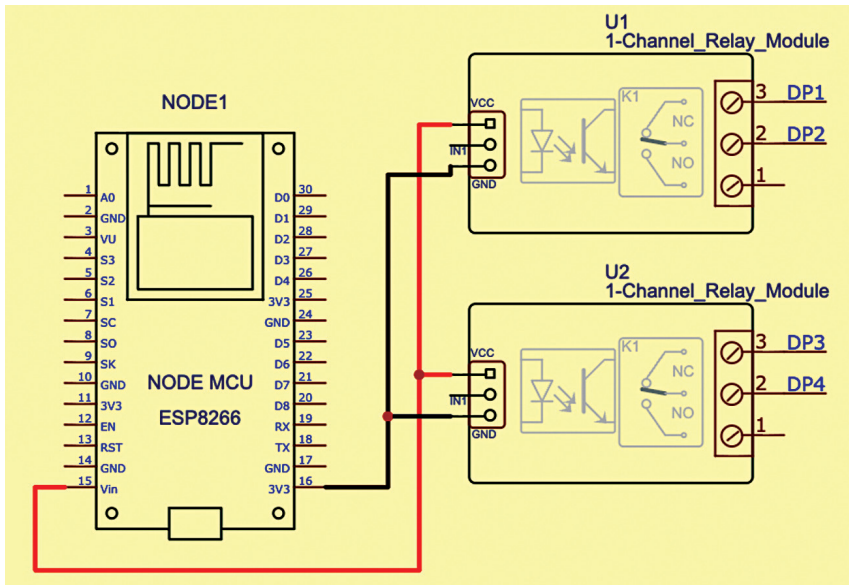


Fig. 4: Load-side circuit

```

3  #include <WiFi.h>
4  #include <Firebase_ESP_Client.h>
5  #include <Wire.h>
6  #include <Adafruit_INA219.h>
7
8  // WiFi credentials
9  #define WIFI_SSID "vivo"
10 #define WIFI_PASSWORD "muru555999"
11
12 // Firebase credentials
13 #define API_KEY "AizaSyCtkhhd_AuunmePvratW6Wk_XXXXXXXXXXXXXXXXXXXXXXXXXXXX"
14 #define DATABASE_URL "https://esp32camsecurity-default-rtdb.asia-southeast1.firebaseio.com/"
15
16 // Sensor pins
17 #define ZMPT_PIN 32
18 #define ACS712_GEN1_PIN 34
19 #define ACS712_GEN2_PIN 35
20 #define RELAY_FAULT_PIN 26
21
22 // Calibration constants
23 #define ZMPT_CALIBRATION 15.26
24 // #define ZMPT_OFFSET 2048
25 #define ZMPT_OFFSET 0
26 #define ACS712_SENSITIVITY 0.185
27 #define ACS712_OFFSET 2500
28
29 // Firebase objects
30 FirebaseData fbd;
31 FirebaseAuth auth;
32 FirebaseConfig config;
33
34 Adafruit_INA219 ina219;
35
36 unsigned long lastSendTime = 0;
37 bool signupOK = false;
38
39 float readACS712Current(int pin) {
40   int raw = analogRead(pin);

```

Fig. 5: Generator-side code snippet configuration (Firestore API)